COMP3028 SOFTWARE CONSTRUCTION

Workshop 9

Guardians of the API Gateway

A beginner-friendly, step-by-step lab

Sessions · JSON Web Tokens · Secure routes

Build two ways to log in, inspect what the browser stores, and decide which users may open a protected route. Every stage includes a concrete action and a result you can check.

Based on: the supplied *Workshop 9: Guardians of the API Gateway* slides. The original three-part sequence and Express libraries are retained. Additional setup, teaching fixtures, verification tasks, and implementation corrections are identified in this guide.

Format: self-contained LaTeX source; compile with pdfLaTeX.
Suggested working time: 150–180 minutes, or two shorter lab sessions.

Contents

1 Start here **2**
1.1 What you will learn . . . 2
1.2 What stays the same, and what this guide adds . . . 2
1.3 Tools and a few words you need . . . 2
1.4 How to follow the instructions . . . 3

2 Prepare your workspace **3**
2.1 Step 1: check the tools . . . 3
2.2 Step 2: obtain the course code . . . 3
2.3 Step 3: create the isolated lab . . . 4
2.4 Step 4: create private configuration . . . 4

3 Part 1: sessions **5**
3.1 Understand the request flow . . . 5
3.2 Step 5: create the demo credential service . . . 5
3.3 Step 6: configure the server . . . 6
3.4 Step 7: add a session demonstration . . . 7
3.5 Step 8: run and inspect the session . . . 7

4 Part 2: JSON Web Tokens **8**
4.1 Understand the difference . . . 8
4.2 Step 9: add token creation and verification . . . 8
4.3 Step 10: create a token from the browser . . . 9
4.4 Step 11: observe a tampered token . . . 9

5 Part 3A: secure the application with sessions **10**
5.1 Step 12: build the login form . . . 10
5.2 Step 13: add session login, role checks, and logout . . . 10
5.3 Step 14: test as a student . . . 12
5.4 Step 15: test as an administrator, then log out . . . 12

6 Part 3B: secure the application with JWTs **13**
6.1 Step 16: add JWT login and authorisation . . . 13
6.2 Step 17: test both roles . . . 14
6.3 Step 18: test tampering and logout . . . 14

7 Verify your completed work **15**
7.1 A repeatable test checklist . . . 15
7.2 Optional terminal tests . . . 15

8 Transfer the work into the course boilerplate **16**

9 Troubleshooting **17**

10 Explain, reflect, and finish **18**
10.1 Questions to answer in your own words . . . 18
10.2 Small practice tasks . . . 18
10.3 Suggested evidence to keep . . . 18
10.4 Local lab versus deployment . . . 19

A Source alignment and corrections **19**

B Reference notes and compiling this guide **20**

1 Start here

1.1 What you will learn

By the end of this workshop, you should be able to:

1. Configure Express sessions and store a small amount of user information on the server.
2. Load application secrets from a local environment file.
3. Create and verify a JSON Web Token (JWT), including its expiry.
4. Authenticate a user through a login form using sessions and then JWTs.
5. Apply middleware to allow or deny access according to a user's role.
6. Explain a failed login, an expired credential, and an authenticated user with insufficient permission.

1.2 What stays the same, and what this guide adds

The original workshop uses the `module-auth` branch of the course repository: `https://github.com/drtaguews/express-boilerplate`. It progresses through **Sessions**, **JWT**, and **Secure Application**. This guide follows the same progression and keeps `express-session`, `cookie-parser`, `dotenv`, `jsonwebtoken`, and the credential-checking method in `UserService`.

The branch could not be retrieved while preparing this guide; its actual folders, database setup, and start scripts have therefore not been assumed. To give everyone an executable learning path, the exercises use a small `workshop9-lab` folder inside your course checkout. It has its own server and two clearly labelled demo accounts. After completing the lab, Section 8 explains how to transfer the same logic into the course boilerplate.

Before you continue

The demo accounts are a teaching fixture. They use plain-text passwords only to isolate the session and JWT lessons. Do not copy this fixture into a deployed application. Use the course application's real credential service and password-hash verification when integrating.

1.3 Tools and a few words you need

Use your course-approved Node.js environment, npm, Git, a code editor, and a browser with developer tools. No database is needed for the isolated lab. You should already be able to create a file, run a terminal command, and read a JavaScript function.

Term	Meaning in this workshop
Request	The browser asking the server to do something, such as open a page or submit a login.
Route	An HTTP method plus a path, such as `POST /login`.
Cookie	A small value the browser stores and sends with matching requests.
Session	Server-side data associated with a browser through a session ID.
JWT	A signed token carrying claims about a user. Signing does not hide its contents.
Authentication	Checking who the user is, normally by validating credentials.

Term	Meaning in this workshop
Authorisation	Checking whether that user is allowed to perform an action.
Middleware	A function that runs before a route handler and can continue or stop the request.

1.4 How to follow the instructions

- Run terminal commands from the folder named in the step. Do not type a leading `$` prompt.
- Save files before restarting the server. Press **Ctrl+C** in the server terminal to stop it.
- Keep a second terminal open for test commands. Keep the server terminal running while testing.
- Use `http://localhost:3000` consistently. Do not switch between `localhost` and `127.0.0.1` while testing cookies.
- Code blocks labelled "append" go above the final error handler and `app.listen()` block. They are additions, not replacements for the entire file.
- The PDF may visually wrap a long code line. A wrapped display line is still part of the same statement.

2 Prepare your workspace

2.1 Step 1: check the tools

Open a terminal and run:

```
node --version
npm --version
git --version
```

Each command should print a version number. If a command is not found, install or enable that tool using your course's setup instructions, reopen the terminal, and try again. You need a supported Node.js release with ES module support; use the same release as your instructor.

2.2 Step 2: obtain the course code

If you do not already have the repository, run:

```
git clone --branch module-auth https://github.com/drtaguews/express-boilerplate.git
cd express-boilerplate
```

If you already have it, open its folder and run `git status`. Save or commit existing work before changing branches. Then run:

```
git fetch origin
git switch module-auth
```

If the branch is not yet local, use `git switch -track origin/module-auth`. If access is denied or the branch is unavailable, ask your instructor for the course copy. You can still complete the isolated lab in any ordinary working folder; do not invent a replacement repository.

2.3 Step 3: create the isolated lab

From the repository root, or your chosen working folder, run:

```
mkdir workshop9-lab
cd workshop9-lab
npm init -y
npm install express express-session cookie-parser dotenv jsonwebtoken ejs
mkdir views
```

What these commands do: create a folder, enter it, create `package.json`, install the required packages, and create a folder for the login page. Keep the generated `package-lock.json` so the installed versions are recorded. If using the supplied companion lab instead, open its folder and run `npm install`; do not repeat `npm init`.

Create these files in your editor:

File	Purpose
`server.mjs`	Express configuration, routes, and middleware
`user-service.mjs`	Two local demo accounts and credential checking
`views/login.ejs`	The reusable login form
`.env`	Local configuration and private secrets
`.gitignore`	Files Git should ignore

The `.mjs` extension makes these JavaScript files ES modules, so the examples can use `import` without changing the parent project's module configuration.

2.4 Step 4: create private configuration

Run this command **twice**. Each run prints a different random value:

```
node -e "console.log(require('node:crypto').randomBytes(32).toString('hex'))"
```

Copy the first value into `SESSION_SECRET` and the second into `JWT_SECRET` in `.env`. Replace the placeholders completely:

```
SESSION_SECRET=PASTE_FIRST_RANDOM_VALUE_HERE
JWT_SECRET=PASTE_SECOND_RANDOM_VALUE_HERE
NODE_ENV=development
PORT=3000
```

Do not add spaces around `=`. Save the file as exactly `.env`, not `.env.txt`. Add the following to `.gitignore` in the lab folder:

```
node_modules/
.env
```

The original slides use short example secrets. Here, you generate your own values instead. `dotenv` loads these settings into `process.env`. Restart Node whenever you change `.env`.

The slides also show `SALT_ROUNDS=1000`. This isolated lab does not hash passwords and does not use that setting. Its meaning depends on the real credential service: do not copy it into a bcrypt cost setting, where a value of 1000 is not an appropriate work factor. Follow the hashing configuration in the course project.

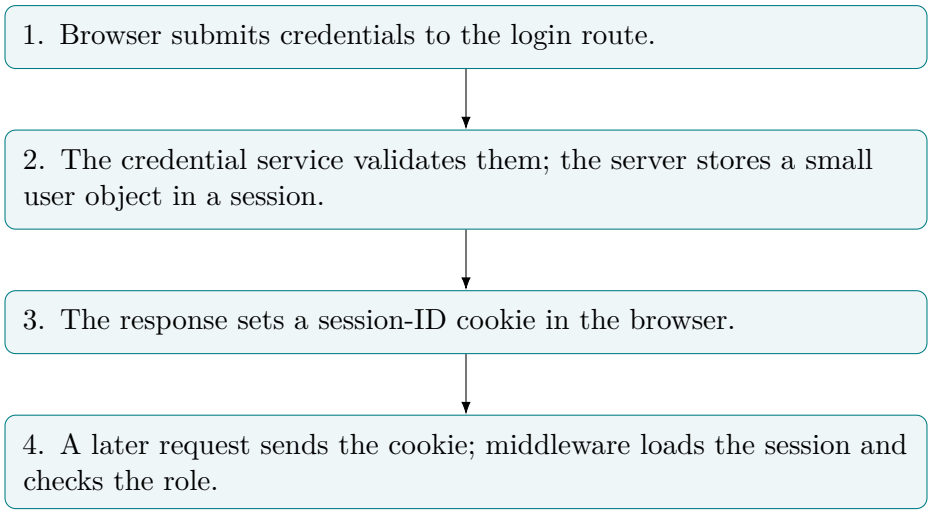
Checkpoint

You are inside `workshop9-lab`, the required packages are installed, and your two secrets are different. If working in Git, `git check-ignore .env` should print `.env`. Never submit your actual secret values.

3 Part 1: sessions

3.1 Understand the request flow

A session lets later requests refer to data stored by the server. The browser carries a session ID, while the server keeps the associated data. In this lab, that storage is memory, so restarting the process loses the sessions.



This is the same login-and-return flow illustrated in the original session authentication diagram. Real deployment assumes HTTPS; the exercises use HTTP on your own computer only.

3.2 Step 5: create the demo credential service

Put the following complete code in `user-service.mjs`. The class name and method mirror the original workshop's service call; the two accounts are additions for this standalone exercise.

```
// Classroom-only fixture, not a real account database.
const users = [
  { id: 1, username: 'brad', password: 'Student123!',
    role: 'student' },
  { id: 2, username: 'alex', password: 'Admin123!',
    role: 'admin' }
];
export class UserService {
  async validUserCredentials(username, password) {
    if (typeof username !== 'string' ||
        typeof password !== 'string' || !username || !password) {
      return null;
    }
    const match = users.find(user =>
      user.username === username && user.password === password
    );
```

```
    if (!match) return null;
    // Do not return the password to the route handler.
    return { id: match.id, username: match.username,
      role: match.role };
  }
}
```

Read the return value: valid credentials return only `id`, `username`, and `role`. Invalid or missing values return `null`. The route must handle both outcomes. Roles come from the server's records, never from a field the user submits.

3.3 Step 6: configure the server

Paste this first block into `server.mjs`:

```
import 'dotenv/config';
import express from 'express';
import session from 'express-session';
import cookieParser from 'cookie-parser';
import jwt from 'jsonwebtoken';
import { fileURLToPath } from 'node:url';
import { UserService } from './user-service.mjs';

for (const key of ['SESSION_SECRET', 'JWT_SECRET']) {
  if (!process.env[key] || process.env[key].length < 32) {
    throw new Error(`${key} must contain at least 32 characters`);
  }
}
const app = express();
const isProduction = process.env.NODE_ENV === 'production';
const cookieOptions = {
  httpOnly: true,
  secure: isProduction,
  sameSite: 'strict',
  path: '/'
};
app.set('view engine', 'ejs');
app.set('views', fileURLToPath(new URL('./views/', import.meta.url)));
app.use(express.urlencoded({ extended: false }));
app.use(express.json());
app.use(cookieParser());
app.use(session({
  name: 'workshop.sid',
  secret: process.env.SESSION_SECRET,
  saveUninitialized: false,
  resave: false,
  cookie: { ...cookieOptions, maxAge: 60000 }
}));
app.use((req, res, next) => {
  res.set('Cache-Control', 'no-store');
  next();
});
app.get('/', (req, res) => {
  res.send('Workshop 9 ready. Visit /login or /loginjwt.');
});
```

Middleware order matters. The form parser must run before routes that read `req.body`. Cookie parsing must run before routes that read `req.cookies`. Session middleware must run before anything reads or writes `req.session`.

Settings to notice: `saveUninitialized: false` avoids saving an untouched session. `resave:`

`false` avoids rewriting unchanged sessions in this lab. `httpOnly` is inside `cookie`; `maxAge: 60000` is a 60-second cookie lifetime. `secure` is false in local development and true in production. `sameSite: 'strict'` limits cross-site cookie sending.

The source installs `cookie-parser` alongside sessions. Modern `express-session` does not require it for its own cookie, but this workshop still needs it to read the JWT cookie in Part 2. The default memory store is suitable only for this exercise; use a deployment-appropriate shared store in a real application.

3.4 Step 7: add a session demonstration

Append this block to `server.mjs`:

```
app.get('/session-demo', (req, res) => {
  req.session.username = 'brad';
  req.session.visits = (req.session.visits || 0) + 1;
  res.json({
    username: req.session.username,
    visits: req.session.visits
  });
});
```

Then append the following final block. Keep it at the bottom when adding later exercises:

```
app.use((err, req, res, next) => {
  console.error(err.message);
  res.status(500).send('Server error. Check the terminal.');
});
const port = Number(process.env.PORT || 3000);
app.listen(port, '127.0.0.1', () => {
  console.log(`Workshop running at http://localhost:${port}`);
});
```

3.5 Step 8: run and inspect the session

1. From `workshop9-lab`, run `node server.mjs`.
2. Check the terminal for `Workshop running at http://localhost:3000`.
3. Open `http://localhost:3000/`. You should see the readiness message. The login links named in it will work after Part 3 is added.
4. Open `http://localhost:3000/session-demo`. Expect `{"username":"brad","visits":1}` (spacing may differ).
5. Refresh promptly. The visit count should increase.
6. Open browser developer tools. In Chrome or Edge, choose **Application** → **Storage** → **Cookies**; in Firefox, choose **Storage** → **Cookies**. Select the localhost origin.
7. Find `workshop.sid`. You should see an opaque ID value, not a JSON object containing the username.
8. Leave the page without refreshing for more than 60 seconds, then request `/session-demo` again. The count should return to 1 because the old cookie has expired.

Checkpoint

Explain why the visit count survives a refresh but resets after expiry. Then restart the server and refresh: the memory-backed session has gone, so the count starts again. A session does not mean permanent storage.

Note on timing: the demo changes session data on every visit, which can renew the cookie. To test expiry, stop making requests during the waiting period. This demo stores a username for learning; it does not authenticate anyone and cannot open the protected dashboard added later.

4 Part 2: JSON Web Tokens

4.1 Understand the difference

A JWT carries signed claims. In this exercise, the server signs a token, the browser stores it in a cookie, and a later request presents it. The server verifies the signature and expiry before trusting its claims. Anyone who has a token may be able to read its payload; do not put passwords or private secrets inside it.

	Session path	**JWT path**
Browser carries	Session ID cookie	Signed token cookie
User data used here	Loaded from server memory	Read from verified claims
Expiry used in lab	60-second session cookie	2-minute demo; 3-minute login token
Logout in this lab	Destroy server session and clear cookie	Clear browser token cookie
Important limitation	Memory data disappears on restart	A copied token may still work until expiry

4.2 Step 9: add token creation and verification

Stop the server with Ctrl+C. Append this block **above** the final error handler and `app.listen()` in `server.mjs`, then restart with `node server.mjs`.

```
function createToken(req, res) {
  // A mock user: this route demonstrates token creation only.
  const user = { id: 1, username: 'brad' };
  const token = jwt.sign(
    { user }, process.env.JWT_SECRET,
    { algorithm: 'HS256', expiresIn: '2m' }
  );
  res.cookie('token', token, cookieOptions);
  res.send('Token created. Now visit /check-token.');
}
function checkToken(req, res) {
  try {
    if (!req.cookies.token) throw new Error('No token');
    const decoded = jwt.verify(
      req.cookies.token, process.env.JWT_SECRET,
      { algorithms: ['HS256'] }
    );
    if (!decoded.user?.username) throw new Error('Wrong payload');
    req.user = decoded.user;
    res.send(`Token is valid. User: ${req.user.username}`);
```

```
  } catch {
    res.status(401).send('Unauthorised access');
  }
}
app.post('/create-token', createToken);
app.get('/check-token', checkToken);
```

The mock user is the same `brad` example used in the original token demonstration. This route issues a token without checking a password, so it is only a learning tool. It deliberately lacks a role and must not grant access to the protected role-based routes.

Important distinction: `jwt.sign()` creates a signature; `jwt.verify()` checks it and validates the expiry. Merely decoding a token is not an authentication check.

4.3 Step 10: create a token from the browser

1. Open `http://localhost:3000/check-token` before creating a token. Expect `Unauthorised access` and status 401.
2. Open developer tools on that localhost page and select **Console**.
3. Type the following expression. It makes a POST request from the same origin and displays the response text. Only run code you understand.

```
fetch('/create-token', { method: 'POST' })
  .then(response => response.text())
  .then(console.log)
```

4. Expect `Token created. Now visit /check-token.`
5. Open `/check-token` in the address bar. Expect `Token is valid. User: brad.`
6. In the Cookies panel, find `token`. Its value normally contains three dot-separated segments. The HttpOnly flag means page JavaScript cannot read that cookie directly.
7. Wait more than two minutes from creation, without issuing a new token, then reload `/check-token`. Expect 401.

Checkpoint

The demo cookie may still exist after two minutes because no cookie lifetime was set on that demo route. Verification must still fail when the token expires. A cookie's lifetime and a JWT's lifetime are separate controls.

4.4 Step 11: observe a tampered token

Create a fresh demo token. In the browser's Cookies panel, replace its value with the literal text `not-a-valid-token`, then visit `/check-token`. Expect 401 again. This shows that possession of a cookie with the right name is not sufficient: the value must verify.

Delete the `token` cookie after this task. Do not upload a real token to an online decoder. The two-minute demo uses `{ user }` as a nested payload; the login token in Part 3 uses top-level `id`, `username`, and `role`. Their verification handlers intentionally read different shapes.

5 Part 3A: secure the application with sessions

5.1 Step 12: build the login form

Create `views/login.ejs` with this complete content:

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title><%= title %></title>
</head>
<body>
  <h1><%= title %></h1>
  <p role="alert"><%= message %></p>
  <form method="post" action="<%= action %>">
    <p><label>Username
      <input name="username" autocomplete="username" required>
    </label></p>
    <p><label>Password
      <input name="password" type="password"
        autocomplete="current-password" required>
    </label></p>
    <button type="submit">Log in</button>
  </form>
  <p><a href="/login">Session login</a> |
     <a href="/loginjwt">JWT login</a></p>
</body>
</html>
```

The `name` attributes determine the keys received in `req.body`. `method="post"` sends credentials in the request body. EJS inserts the title, action, and message provided by `res.render()`; the escaped `<%= ... %>` form is used for those values.

5.2 Step 13: add session login, role checks, and logout

Stop the server. Insert the next block above the final error handler and listener in `server.mjs`. Restart the server afterwards.

```
app.get('/login', (req, res) => {
  res.render('login', {
    title: 'Session login', action: '/login', message: ''
  });
});
app.post('/login', async (req, res, next) => {
  try {
    const { username, password } = req.body || {};
    const user = await new UserService()
      .validUserCredentials(username, password);
    if (!user) {
      return req.session.destroy(err => {
        if (err) return next(err);
        res.clearCookie('workshop.sid', cookieOptions);
        return res.status(401).render('login', {
          title: 'Session login', action: '/login',
          message: 'Invalid username or password'
        });
      });
    }
```

```
    // Issue a new session ID after successful authentication.
    req.session.regenerate(err => {
      if (err) return next(err);
      req.session.user = {
        id: user.id, username: user.username, role: user.role
      };
      req.session.save(err => {
        if (err) return next(err);
        return res.redirect(302, '/dashboard');
      });
    });
  } catch (err) {
    next(err);
  }
});
function authoriseSession(roles) {
  return (req, res, next) => {
    const user = req.session.user;
    if (!user) return res.redirect(302, '/login');
    if (!roles.includes(user.role)) {
      return res.status(403).send('Forbidden: role not allowed');
    }
    next();
  };
}
app.get('/dashboard', authoriseSession(['student', 'admin']),
  (req, res) => res.json({
    message: 'Session dashboard', user: req.session.user
  })
);
app.get('/admin', authoriseSession(['admin']),
  (req, res) => res.send('Session admin area')
);
app.post('/logout', (req, res, next) => {
  req.session.destroy(err => {
    if (err) return next(err);
    res.clearCookie('workshop.sid', cookieOptions);
    res.redirect(302, '/login');
  });
});
```

Read the login handler in small pieces

1. `GET /login` displays the form. `POST /login` receives the submitted values.
2. The service returns a user or `null`. A missing field is also invalid. Invalid credentials clear any existing session and show the same general error message.
3. On success, `regenerate()` changes the session ID. This prevents carrying an old session ID into the authenticated state.
4. Only the small user object is stored. The password is not saved in the session.
5. `save()` finishes before the redirect so the next request can read the authenticated session.
6. The browser follows the 302 redirect to `/dashboard`.

Read the role middleware

`authoriseSession(['admin'])` returns a middleware function. If no user is logged in, it redirects to the login form. If the user is logged in but has the wrong role, it returns 403. Otherwise, `next()` allows the route handler to run. Returning after a denial prevents the handler from continuing accidentally.

The original slide redirects both denied cases to login. This guide separates them so students can see the difference between "not logged in" and "logged in, but not allowed". The role check is unchanged in purpose.

5.3 Step 14: test as a student

1. Delete `workshop.sid` in developer tools, or use a fresh private browser window.
2. Open `/dashboard`. You should end up at `/login`.
3. Enter username `brad` and password `Student123!`. Submit the form.
4. Expect the dashboard JSON to show `Session dashboard` and the user role `student`.
5. Immediately open `/admin`. Expect `Forbidden: role not allowed` with status 403.
6. Visit `/login`, enter an incorrect password, and submit. Expect the error message and a 401 response. The earlier authenticated session is cleared.

Browser developer tools' **Network** tab shows HTTP statuses. Enable **Preserve log** if you want to see the 302 response before the browser follows its redirect. The final page may show 200 even though the first response was 302.

5.4 Step 15: test as an administrator, then log out

1. Visit `/login`. Enter `alex` and `Admin123!`.
2. Visit `/admin` within 60 seconds. Expect `Session admin area`.
3. On a localhost page, run this in the browser console:

```
fetch('/logout', { method: 'POST' })
  .then(() => { window.location.href = '/dashboard'; })
```

You should arrive at the login form. Logout destroys the session data and clears the cookie. A fetch follows a redirect internally but does not navigate the browser tab, which is why the expression explicitly changes `window.location.href`.

Checkpoint

Log in again, then make no requests for more than 60 seconds. Open `/dashboard`; you should be asked to log in. If it expires while you are reading instructions, log in again. The short lifetime is intentional for the exercise.

6 Part 3B: secure the application with JWTs

6.1 Step 16: add JWT login and authorisation

Stop the server. Insert this block before the final error handler and listener, then restart. The JWT path uses different dashboard URLs so you can compare both approaches in one lab.

```
app.get('/loginjwt', (req, res) => {
  res.render('login', {
    title: 'JWT login', action: '/loginjwt', message: ''
  });
});
app.post('/loginjwt', async (req, res, next) => {
  try {
    const { username, password } = req.body || {};
    const user = await new UserService()
      .validUserCredentials(username, password);
    if (!user) {
      res.clearCookie('token', cookieOptions);
      return res.status(401).render('login', {
        title: 'JWT login', action: '/loginjwt',
        message: 'Invalid username or password'
      });
    }
    const token = jwt.sign(
      { id: user.id, username: user.username, role: user.role },
      process.env.JWT_SECRET,
      { algorithm: 'HS256', expiresIn: '3m' }
    );
    res.cookie('token', token, {
      ...cookieOptions, maxAge: 180000
    });
    res.redirect(302, '/dashboardjwt');
  } catch (err) {
    next(err);
  }
});
function authoriseJWT(roles) {
  return (req, res, next) => {
    const token = req.cookies.token;
    if (!token) return res.redirect(302, '/loginjwt');
    let user;
    try {
      user = jwt.verify(token, process.env.JWT_SECRET, {
        algorithms: ['HS256']
      });
    } catch {
      res.clearCookie('token', cookieOptions);
      return res.redirect(302, '/loginjwt');
    }
    if (!roles.includes(user.role)) {
      return res.status(403).send('Forbidden: role not allowed');
    }
    req.user = user;
    next();
  };
}
app.get('/dashboardjwt', authoriseJWT(['student', 'admin']),
  (req, res) => res.json({
    message: 'JWT dashboard', user: req.user
  })
);
app.get('/adminjwt', authoriseJWT(['admin']),
```

```
  (req, res) => res.send('JWT admin area')
);
app.post('/logoutjwt', (req, res) => {
  res.clearCookie('token', cookieOptions);
  res.redirect(302, '/loginjwt');
});
```

Follow the data

1. The same credential service validates the submitted username and password.
2. `jwt.sign()` receives only the identity and role, plus the signing secret and token options.
3. The token is stored in the browser's `token` cookie. The signature protects the claims against undetected modification; it does not encrypt them.
4. `authoriseJWT()` verifies the token, checks the role, assigns `req.user`, and calls `next()` only if access is allowed.
5. The route reads `req.user`. It does not rely on `req.session.user`.

The original JWT login slide puts `expiresIn: '3m'` inside `res.cookie()`. That does not set a JWT expiry. Here it is correctly passed to `jwt.sign()`, and the browser cookie gets its own `maxAge: 180000` in milliseconds. The two-minute token demonstration and the three-minute login token preserve the timings shown in the slides.

6.2 Step 17: test both roles

1. Delete the `token` cookie. Open `/dashboardjwt`; expect a redirect to `/loginjwt`.
2. Log in as `brad` with `Student123!`. Expect dashboard JSON showing `JWT dashboard`, username `brad`, and role `student`. Extra `iat` and `exp` claims are normal.
3. Visit `/adminjwt`. Expect 403.
4. Visit `/loginjwt` and log in as `alex` with `Admin123!`. Visit `/adminjwt`; expect `JWT admin area`.
5. Wait more than three minutes after login, without logging in again. Visit `/dashboardjwt`; expect the login form.

Do not mix token demos: after JWT login, `/check-token` is no longer the right test route because it expects the nested demo payload. Likewise, calling `/create-token` overwrites the login token with a demo token that lacks a role. Use `/dashboardjwt` to test the login flow.

6.3 Step 18: test tampering and logout

Log in again. Change the `token` cookie to `not-a-valid-token` and open `/dashboardjwt`. The verification failure should clear the cookie and send you to login. Next, log in and run:

```
fetch('/logoutjwt', { method: 'POST' })
  .then(() => { window.location.href = '/dashboardjwt'; })
```

You should see the login form again. Clearing this browser's cookie does *not* revoke every copy of that token. Someone holding a previously copied valid token could still present it until expiry. Immediate revocation needs an additional design, beyond this lab.

Checkpoint

A session login must not automatically authorise `/dashboardjwt`; a JWT login must not automatically authorise `/dashboard`. Clear both cookies before checking this. Each route must use its own credential and role-checking middleware.

7 Verify your completed work

7.1 A repeatable test checklist

Use separate fresh logins when needed; short expiry times can otherwise confuse your results. Record the route, the account, the status or redirect, and what you saw.

Test	Action	Expected result
1	Open `/session-demo`, then refresh promptly.	Visit count increases.
2	Wait over 60 seconds without requests, then open the session demo.	Count starts at 1 again.
3	Open `/check-token` with no token.	401 Unauthorised access.
4	POST to `/create-token`, then open `/check-token`.	200; user is brad.
5	Wait over two minutes after demo token creation, then check it.	401, even if the cookie remains.
6	Open either dashboard with its credential absent.	302 to the corresponding login form.
7	Submit an incorrect password to each login route.	401; generic invalid-credentials message.
8	Submit a login request missing a field.	401; no successful login.
9	Log in as brad and open the matching dashboard.	200; role is student.
10	As brad, open the matching admin route.	403 Forbidden.
11	Log in as alex and open the matching admin route.	200; admin-area message.
12	Replace the JWT cookie with invalid text; open JWT dashboard.	302 to JWT login; cookie cleared.
13	Let a login credential expire, then open its dashboard.	Redirect to its login form.
14	POST to matching logout route, then open its dashboard.	Redirect to login.
15	Restart Node after a session login.	Previous session cannot open dashboard.
16	Restart Node after JWT login, keeping the same secret and within expiry.	JWT can still open its dashboard.

7.2 Optional terminal tests

These are alternatives to browser tests. They use their own cookie files; browser cookies are not shared with curl. In Windows PowerShell, use `curl.exe` in place of `curl` if `curl` invokes a PowerShell alias. Run each command on one line in a second terminal.

```
curl -i http://localhost:3000/dashboard
curl -i -c session-cookies.txt -d "username=brad&password=Student123!" http://localhost:3000/
    login
curl -i -b session-cookies.txt http://localhost:3000/dashboard
curl -i -b session-cookies.txt http://localhost:3000/admin
curl -i -b session-cookies.txt -c session-cookies.txt -X POST http://localhost:3000/logout
curl -i -c jwt-cookies.txt -d "username=alex&password=Admin123!" http://localhost:3000/loginjwt
curl -i -b jwt-cookies.txt http://localhost:3000/adminjwt
curl -i -d "username=brad" http://localhost:3000/login
```

`-i` displays response headers, `-c` saves cookies, `-b` sends saved cookies, and `-d` submits form data as a POST request. These commands do not follow redirects automatically, so a 302 response is visible. Delete the temporary cookie files when finished and do not submit them. The example passwords belong only to the local demo accounts.

8 Transfer the work into the course boilerplate

Complete this stage using the instructor-provided `module-auth` checkout. Exact paths vary; use the names and structure that actually exist in your copy.

1. **Run the original application first.** Read its README, inspect `package.json`, and use its documented install, database, seed, and start commands. Do not assume `node server.mjs` starts the original project.
2. **Find the Express configuration.** Search the editor for `express()` and `app.use`. Add missing parsers and session middleware once, before protected routes. Reuse existing imports and avoid a second listener.
3. **Find the real credential service.** Search for `class UserService` or `validUserCredentials`. Import it using its actual path and module style. Do not replace it with the lab fixture.
4. **Check its contract.** Determine whether invalid credentials return `null`/`false` or throw a documented authentication error. Handle invalid credentials with a generic login message; pass unexpected database or infrastructure failures to the error handler.
5. **Check its user fields.** Map the verified record to `{id, username, role}`. Use your actual role strings consistently; do not assume the course data contains `student` and `admin`.
6. **Add the controllers.** Move the GET/POST session and JWT login logic into the appropriate controller files. Preserve the existing view engine and adapt `res.render('login', ...)` to the template already used.
7. **Add the middleware.** Put the session and JWT role checks in separate middleware modules. Export and import them using the project's established style.
8. **Wire the routes.** Put the chosen authorisation middleware before each protected handler. Keep login and necessary public assets accessible. Retain distinct routes while comparing both approaches, or use the original `/dashboard` for one approach at a time.
9. **Retest with course accounts.** Use authorised seeded/test accounts and the actual roles. Repeat missing credential, bad password, allowed role, denied role, expiry, and logout checks.
10. **Remove demonstrations from deployed code.** Do not expose `/create-token`, `/session-demo`, or the demo credential fixture. They are teaching aids, not part of the real authentication flow.

The original profile example reads `req.session.username`. Once login stores the structured `req.session.user`, read from that same location instead:

```
// Attach authoriseSession(...) before this controller.
export function profile(req, res) {
  res.render('home', {
    username: req.session.user.username
  });
}
```

For a JWT-protected profile, use `req.user.username` after JWT middleware has verified and populated it. Adapt the template name to your project.

9 Troubleshooting

Symptom	Check and fix
Command not found	Check Node.js/npm/Git installation and reopen the terminal. Run the three version checks again.
Cannot find a package	Run `npm install` in the lab folder containing `package.json`, then restart.
Import syntax error	Use `server.mjs` and `user-service.mjs`, including extensions in the import. Do not rename them to `.js` unless the project is configured for ES modules.
Missing or short secret	Confirm `.env` is in the terminal's working directory, contains the exact key names, and has the generated values. Restart Node.
EADDRINUSE	Another process is using port 3000. Stop your earlier lab server or change `PORT` and use that port consistently in every URL.
Cannot GET /create-token	That route uses POST. Use the console fetch command; typing the URL sends GET.
Cannot GET /login	Add the Part 3 session routes, save, and restart. Routes from later stages do not exist in the initial server.
Failed to look up view	Check `views/login.ejs`, the exact filename, and the EJS installation.
Login always fails	Check exact username/password spelling and case. Form fields must be named `username` and `password`. Use the demo accounts only with the demo service.
req.body is undefined	Put `express.urlencoded(...)` before form routes. For JSON clients, also keep `express.json()` before routes.
No cookie on local HTTP	Keep `NODE_ENV=development`. A secure cookie requires HTTPS. Use the same hostname throughout.
Immediate return to login	Check cookie existence, expiry, middleware order, and which login path you used. Log in again after restarting a memory-session server.
403 after login	Authentication succeeded, but the role is not in the allowed list. Check the verified user's role and the middleware array.
/check-token fails after JWT login	That demonstration expects a nested payload. Test the login token using `/dashboardjwt` instead.

Symptom	Check and fix
JWT changed but old permissions persist	A signed token contains the role at issuance. Log in again to issue new claims; real-time role changes need an additional design.
Headers already sent	Use `return` after response/redirect branches; do not call `next()` after sending a denial.
.env appears in Git	Add the ignore rule. Ignoring does not untrack an already committed file. Remove it from tracking and rotate any exposed secrets following instructor guidance.

10 Explain, reflect, and finish

10.1 Questions to answer in your own words

1. What does the browser store for a session? Where does the username live?
2. Why is a readable JWT payload not evidence that a token is valid?
3. Why must the role come from the credential service rather than a login form field?
4. What is the difference between a redirect to login and a 403 response?
5. Why is `expiresIn` passed to `jwt.sign()` rather than `res.cookie()`?
6. What happens to each authentication method when the server restarts? Explain the importance of the signing secret remaining the same for JWTs.
7. Why does deleting a JWT cookie not revoke a stolen copy of the same token?
8. What would break if you placed a global authorisation middleware before the login route?

10.2 Small practice tasks

1. Add `GET /reports` for session-authenticated administrators only. Reuse the existing role middleware. Test it as brad, alex, and a logged-out visitor.
2. Add the equivalent JWT route, `GET /reportsjwt`, and repeat the same tests.
3. Change the JWT login lifetime to 30 seconds and the cookie lifetime to 30000 milliseconds. Verify expiry, then restore the workshop values.
4. Explain how route-based protection lets the home page remain public while dashboards remain private. Do not protect only the navigation link: protect the server route itself.

10.3 Suggested evidence to keep

Unless your instructor gives different requirements, keep the completed source, the filled test checklist, and short answers to the reflection questions. Capture one successful login, one denied role, and one expired credential. Hide cookie values and secrets in screenshots. This is a suggested learning record, not a replacement for the course's marking requirements.

10.4 Local lab versus deployment

The original workshop assumes HTTPS. Before applying this pattern to a real system, replace the fixture with proper password verification, use HTTPS and an appropriate session store, and add protections for login abuse and state-changing requests. HttpOnly does not stop every script-related attack, and SameSite is not a complete substitute for a considered CSRF defence. If HTTPS terminates at a reverse proxy, configure Express's proxy trust to match the actual infrastructure; do not enable broad proxy trust blindly.

These deployment concerns do not change the workshop's central lesson: validate credentials, establish a session or token, and enforce access checks on the server before a protected handler runs.

A Source alignment and corrections

Original material	Treatment in this guide
Boilerplate, printed p. 4	Same repository and `module-auth` branch. A separate runnable lab is added because the repository layout was not verified.
Sessions, pp. 6–9	Same libraries, environment-secret idea, session request property, and 60000 ms timing; expanded setup and inspection steps.
Session options	Move `httpOnly` into `cookie`; use `resave: false` for the exercise. Keep `saveUninitialized: false`.
Secrets	Replace hard-coded demonstration secrets with generated values; explain why the original salt setting is not copied without its hashing context.
JWT demo, pp. 11–13	Keep mock brad user, nested payload, cookie, verification, and two-minute lifetime. Add explicit routes and tamper/expiry checks.
Authentication, pp. 16–19	Keep credential service, small user object, cookie delivery, and dashboard redirects. Add a complete form, explicit invalid-user branch, and session regeneration.
JWT login expiry	Put the original three-minute expiry in signing options; set browser cookie lifetime separately.
Authorisation, pp. 20–23	Retain middleware factories and role inclusion checks. Add route wiring and show 403 for a valid user with insufficient permission.
Added teaching support	Two fixture accounts, distinct JWT dashboard routes, logout, expected outputs, troubleshooting, optional curl tests, and integration guidance.

B Reference notes and compiling this guide

1. *COMP3028 Software Construction, Workshop 9: Guardians of the API Gateway.* User-supplied PDF; primary source for the workshop content and sequence.
2. Course boilerplate (link provided in the source):
   `https://github.com/drtaguews/express-boilerplate`
   Branch: `module-auth`. Repository contents were not available for verification during preparation.
3. Express session middleware reference, consulted for option placement and session behaviour:
   `https://expressjs.com/en/resources/middleware/session/`
4. Auth0, *node-jsonwebtoken*, consulted for token signing, verification, and expiry options:
   `https://github.com/auth0/node-jsonwebtoken`

Overleaf: create a blank project, upload this `.tex` file, set it as the main document, choose **pdfLaTeX**, and recompile. The source embeds all code listings and draws its diagram with TikZ; it requires no external images or shell-escape setting.

Local compilation:

```
pdflatex Workshop_9_Beginner_Guide.tex
pdflatex Workshop_9_Beginner_Guide.tex
```

Run twice so the contents page and cross-references update. If you receive a missing-package error, install the named LaTeX package or use a full TeX distribution. The JavaScript lab runs in a terminal with Node.js; it does not run inside Overleaf.